Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

AI Vibe Coding for SQL

Course Code: C1154

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	27 August 2026	Initial AI Vibe Coding for SQL release with 11 browser SQLite labs, aligned slides and AI prompt validation controls.	Tertiary Infotech Academy Pte Ltd

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

Right-click and choose “Update Field”, or press F9, to build the table of contents.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Course Overview

This guide contains the complete step-by-step workflow for AI-assisted SQL modelling, querying, transformation and delivery. The slide deck explains mechanisms and evidence; this guide and the lab folders contain the procedures.

AI-Assisted SQL Safety Contract

- Supply the SQLite dialect, schema, business rule, expected output and edge cases.
- Require assumptions and uncertainties to be explicit.
- Use bound parameters for values and allow-list any dynamic structure.
- Execute generated writes inside a transaction and test rollback.
- Reconcile row counts, amounts and keys before approving results.

Learning Outcomes

- LO1: Apply data modelling.
- LO2: Apply data processing and analysis.
- LO3: Apply data aggregation and transformation.
- LO4: Perform data mapping to a data warehouse.

Topic 01 — AI-Assisted SQL Data Modelling and Database Design

Requirements to normalised relational schemas, constraints and safe AI prompt contracts

Requirement contract — Entities, attributes, relationships, invariants and sample rows become explicit prompt inputs.

Relational integrity — PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE and CHECK constraints reject invalid states.

Normalisation — 1NF removes repeating groups; 2NF removes partial dependency; 3NF removes transitive dependency.

AI validation gate — Treat generated DDL as a proposal: inspect, execute in a transaction, test constraints, then approve.

Lab 1 — Prompt-to-Schema Design Studio

Model a Singapore retail ordering workflow from an ambiguous business brief.

AI prompt

Act as a senior SQLite data modeller. From the requirements below, return: assumptions; entities and cardinalities; 3NF tables; SQLite DDL with explicit PK, FK,

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

NOT NULL, UNIQUE and CHECK constraints; five adversarial test cases. Do not invent fields. Mark every uncertainty as NEEDS_CONFIRMATION.

Detailed step-by-step

1. Read REQUIREMENTS.md and identify entities, events, rules and unknowns.
2. Paste the supplied prompt into an approved AI assistant and review its assumptions before any DDL.
3. Edit schema.sql so every business invariant has a SQLite constraint or an explicit application check.
4. Start the local server, open the app and run the schema-inspection query.
5. Execute the adversarial inserts in tests.sql and record which constraint blocks each invalid row.
6. Save the final prompt, corrected DDL and validation evidence in evidence.md.

Core SQL

```
SELECT name, sql FROM sqlite_master WHERE type='table' ORDER BY name;
```

Acceptance and troubleshooting

Acceptance: Invalid email, negative price and orphan order rows are rejected; valid sample rows insert successfully.

If it fails: AI invents fields or omits constraints → restate the data contract and require a requirement-to-column trace.

Lab 2 — SQLite To-Do List Web App

Build a personal work queue with priorities, due dates and completion state.

AI prompt

Generate SQLite DDL and parameterised CRUD statements for a tasks table with title, priority, due_date, completed and created_at. Enforce allowed priorities and completed as 0/1. Return SQL only, then list five tests separately.

Detailed step-by-step

7. Serve the folder with python3 -m http.server 8000; do not open index.html directly.
8. Open the app and inspect the seeded task list and status counters.
9. Add three tasks covering all priorities and one blank-title attempt.
10. Toggle one task complete and delete another; verify the counters and sorted list change.
11. Open the SQL console and run the supplied analysis query.
12. Export todo.db, refresh the browser, import the file and verify row-level persistence.

Core SQL

```
SELECT id,title,priority,due_date,completed FROM tasks ORDER BY completed, CASE
priority WHEN 'High' THEN 1 WHEN 'Medium' THEN 2 ELSE 3 END, due_date;
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Acceptance and troubleshooting

Acceptance: Add, complete and delete tasks; export the database, reload the page, import it and recover the same rows.

If it fails: Unsafe string concatenation or missing domain checks → replace with bound parameters and CHECK constraints.

Lab 3 — Customer and Order Relationship Lab

Turn spreadsheet-style customer and order data into a clean relational model.

AI prompt

Review these customer/order columns. Identify repeating groups and dependencies, propose a 3NF SQLite design, state the grain of each table, and provide DDL plus orphan-key tests. Preserve source identifiers for reconciliation.

Detailed step-by-step

13. Profile the flat source rows and mark repeating customer/product values.
14. Use the AI prompt to propose tables, but reject any entity not supported by the source.
15. Create schema.sql with foreign_keys enabled and explicit ON DELETE behaviour.
16. Load seed.sql in the browser app and inspect table row counts.
17. Run the three-table revenue join and compare the total to CONTROL_TOTALS.md.
18. Attempt an orphan order item and capture the foreign-key failure in evidence.md.

Core SQL

```
SELECT c.name, COUNT(DISTINCT o.id) orders, ROUND(SUM(oi.qty*oi.unit_price),2)
revenue FROM customers c JOIN orders o ON o.customer_id=c.id JOIN order_items
oi ON oi.order_id=o.id GROUP BY c.id,c.name ORDER BY revenue DESC;
```

Acceptance and troubleshooting

Acceptance: The revenue total matches the seed control total and every order item resolves to one order and one product.

If it fails: Join duplicates inflate revenue → aggregate at order-item grain and verify keys before summing.

Topic 02 — SQL Data Processing, Querying and Analysis

CRUD, predicates, joins, parameter binding and evidence-led correction of AI-generated queries

Query pipeline — FROM/JOIN builds the rowset; WHERE filters; GROUP BY aggregates; HAVING filters groups; SELECT projects; ORDER BY sorts.

Safe parameters — Bind user input as values instead of concatenating strings; structure such as column names stays allow-listed.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Join cardinality — Keys and relationship direction predict row counts; unexpected multiplication is a diagnostic signal.

CRUD control — Validate inputs, wrap dependent writes in a transaction, verify affected rows and preserve an audit trail.

Lab 4 — Inventory CRUD Web App

Manage stock levels for a small Singapore retailer without allowing negative inventory.

AI prompt

Produce parameterised SQLite CRUD statements for products. A stock adjustment must be atomic, reject negative resulting stock, and return the updated row. Include a transaction and rollback path.

Detailed step-by-step

19. Open the inventory app and filter products by name or SKU.
20. Add a new product using a unique SKU and positive price.
21. Adjust stock upward, then downward within the available balance.
22. Attempt a withdrawal that would make stock negative and verify no row changes.
23. Run the low-stock query and compare badges with query results.
24. Export the database and save an evidence screenshot plus the final SQL prompt.

Core SQL

```
SELECT sku,name,stock,reorder_level, CASE WHEN stock<=reorder_level THEN
'REORDER' ELSE 'OK' END status FROM products ORDER BY status DESC, stock;
```

Acceptance and troubleshooting

Acceptance: Positive and negative adjustments update one row; an adjustment below zero is rolled back.

If it fails: Read-modify-write race or negative balance → use one guarded UPDATE inside a transaction and check changes().

Lab 5 — Order Query and Filter Explorer

Answer operations questions using SELECT, WHERE, ORDER BY, LIMIT and date filters.

AI prompt

Given this schema and business question, generate one SQLite SELECT using named parameters. Explain predicate selectivity and null behaviour. Provide a second query that proves the result is complete.

Detailed step-by-step

25. Inspect the orders data dictionary and the business questions in TASKS.md.
26. Prompt the AI for a parameterised query and challenge its assumptions about nulls and dates.
27. Run the query with the supplied values in the app console.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

28. Change one parameter at a time and predict the row-count change before executing.
29. Run the control COUNT query and reconcile filtered rows.
30. Save the corrected query set and a short explanation of each predicate.

Core SQL

```
SELECT order_no,customer_name,status,total,order_date FROM orders WHERE
status='Paid' AND order_date>='2026-08-01' ORDER BY total DESC LIMIT 10;
```

Acceptance and troubleshooting

Acceptance: Results satisfy every predicate, sort descending and match an independent COUNT control query.

If it fails: NULL rows disappear unexpectedly → state three-valued logic and test IS NULL explicitly.

Lab 6 — Service Desk Join Lab

Combine tickets, requesters and agents without losing unassigned tickets.

AI prompt

Write a SQLite query for all tickets with requester and optional assigned agent. Preserve unassigned tickets, derive age_hours, and order SLA breaches first. Explain why each join is INNER or LEFT.

Detailed step-by-step

31. Review the ticket relationship diagram and predict the result cardinality.
32. Create two tickets, leaving one unassigned.
33. Use the AI prompt to generate the report and inspect its join choices.
34. Run the query and reconcile output rows to the ticket count.
35. Assign an agent and close a ticket through the interface.
36. Record before/after queries and explain why the LEFT JOIN is required.

Core SQL

```
SELECT t.id,t.subject,r.name requester,a.name agent,ROUND((julianday('now')-
julianday(t.created_at))*24,1) age_hours FROM tickets t JOIN people r ON
r.id=t.requester_id LEFT JOIN people a ON a.id=t.agent_id ORDER BY
(age_hours>24) DESC,age_hours DESC;
```

Acceptance and troubleshooting

Acceptance: Every ticket appears exactly once and unassigned tickets show a blank agent rather than disappearing.

If it fails: INNER JOIN removes optional relationships → change only the optional side to LEFT JOIN and test row counts.

Topic 03 — SQL Data Aggregation and Transformation

Aggregate measures, CTEs, data-quality rules, transactions, indexes and query plans

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Grain first — Define what one output row represents before choosing GROUP BY columns or joining detail tables.

Quality profile — Count nulls, duplicates, invalid domains and orphan keys before transforming source data.

Atomic change — BEGIN, COMMIT and ROLLBACK protect multi-step updates from partial completion.

Performance evidence — EXPLAIN QUERY PLAN proves whether an index changes access from SCAN to SEARCH.

Lab 7 — Sales Aggregation Dashboard

Create management KPIs from order-line data at a declared grain.

AI prompt

At order-line grain, generate SQLite queries for revenue, distinct orders, average order value and monthly category totals. Prevent double counting and define every numerator and denominator.

Detailed step-by-step

37. Write the metric definitions before writing SQL.
38. Prompt the AI with the declared grain and required reconciliations.
39. Run the base line-grain query and inspect sample rows.
40. Run the KPI and category queries and compare them with dashboard cards.
41. Introduce a duplicate join deliberately and observe inflated revenue.
42. Fix the join, record both plans and save the reconciled metric table.

Core SQL

```
WITH line AS (SELECT oi.order_id,p.category,oi.qty*oi.unit_price amount FROM
order_items oi JOIN products p ON p.id=oi.product_id) SELECT
category,ROUND(SUM(amount),2) revenue,COUNT(DISTINCT order_id) orders FROM line
GROUP BY category ORDER BY revenue DESC;
```

Acceptance and troubleshooting

Acceptance: Dashboard grand total equals SUM(qty*unit_price) from order_items and AOV equals revenue/distinct orders.

If it fails: Averages use line count instead of order count → declare grain and use COUNT(DISTINCT order_id).

Lab 8 — Data Quality and Transaction Control

Clean supplier data and update linked records without partial writes.

AI prompt

Profile this SQLite supplier table for nulls, duplicate emails, invalid status values and orphan product references. Propose deterministic corrections, a reject table and one atomic transaction. Never delete source rows silently.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Detailed step-by-step

43. Run the profiling queries and record issue counts.
44. Review AI-suggested corrections; reject any rule that guesses missing facts.
45. Create rejected_rows with source key, reason and captured_at.
46. Execute the correction transaction and deliberately trigger one failure.
47. Verify ROLLBACK restores every source row.
48. Run reconciliation totals and document approved versus quarantined rows.

Core SQL

```
SELECT 'null_email' issue,COUNT(*) n FROM suppliers WHERE email IS NULL UNION ALL SELECT 'duplicate_email',COUNT(*) FROM (SELECT email FROM suppliers WHERE email IS NOT NULL GROUP BY email HAVING COUNT(*)>1);
```

Acceptance and troubleshooting

Acceptance: The transaction either applies all linked corrections or rolls back; source and reject totals reconcile.

If it fails: AI overwrites evidence or applies non-deterministic fixes → retain raw values, reason codes and explicit rules.

Lab 9 — CTEs, Indexes and Query Plans

Improve a slow order-history query using evidence rather than intuition.

AI prompt

Refactor this SQLite query into named CTEs, then propose the smallest useful index. Explain column order from equality, range and ordering predicates. Require EXPLAIN QUERY PLAN evidence before claiming improvement.

Detailed step-by-step

49. Run the target query and save its ordered result set.
50. Capture the baseline EXPLAIN QUERY PLAN output.
51. Ask the AI for one justified composite index, not a list of speculative indexes.
52. Create the index and capture the new plan.
53. Compare results and measure execution across repeated runs.
54. Document the access trade-off: faster reads versus write/storage cost.

Core SQL

```
EXPLAIN QUERY PLAN SELECT customer_id,order_date,total FROM orders WHERE customer_id=7 AND order_date>='2026-01-01' ORDER BY order_date DESC;
```

Acceptance and troubleshooting

Acceptance: Plan changes from full SCAN to indexed SEARCH and the result set remains byte-for-byte equivalent.

If it fails: Index adds storage but is unused → compare plan, predicate order and selectivity; remove redundant index.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Topic 04 — Data Mapping, Warehousing and Dashboard Development

Source-to-target mapping, star schemas, dimensional loads and browser dashboards backed by SQLite

Warehouse grain — A fact table states one measurable event; dimensions provide stable descriptive context.

Source-to-target map — Each target column records source, transform, data type, null rule and reconciliation test.

Load discipline — Load dimensions before facts, resolve surrogate keys, quarantine rejects and reconcile totals.

Dashboard contract — Each visual owns a metric definition, SQL query, filter behaviour and acceptance threshold.

Lab 10 — Warehouse Star Schema Dashboard

Map retail transactions into a small analytical star schema.

AI prompt

Design a SQLite star schema for retail sales at order-line grain. Return source-to-target mappings, surrogate-key strategy, dimension-first load SQL, reject handling and three reconciliation queries. Distinguish additive from non-additive measures.

Detailed step-by-step

55. Declare the fact grain and identify measures versus dimension attributes.
56. Use the AI prompt to draft mappings, then trace every target field to a source expression.
57. Run schema.sql and load dimensions before fact_sales.
58. Execute row-count, amount and key-reconciliation queries.
59. Open the warehouse dashboard and compare chart totals to SQL output.
60. Save mapping.csv, reconciliation.md and a screenshot of the final dashboard.

Core SQL

```
SELECT d.month_name,p.category,ROUND(SUM(f.sales_amount),2) sales FROM
fact_sales f JOIN dim_date d ON d.date_key=f.date_key JOIN dim_product p ON
p.product_key=f.product_key GROUP BY d.month_name,p.category ORDER BY
d.date_key,p.category;
```

Acceptance and troubleshooting

Acceptance: Fact row count equals accepted source lines; fact amount reconciles to source amount; no unknown foreign keys remain.

If it fails: Facts load before dimensions or grain is unclear → resolve dimension keys first and reject unresolved rows.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 11 — Capstone: AI-Built Service Operations App

Deliver a small service-operations product from brief to validated dashboard.

AI prompt

You are my SQL pair programmer. Work in stages and wait for approval after each: clarify requirements; propose 3NF model; generate SQLite DDL; generate parameterised CRUD; generate SLA report; generate tests and threat review. For every stage, list assumptions, expected outputs and failure tests. Do not change approved contracts silently.

Detailed step-by-step

61. Read the product brief and write five clarifying questions before using AI.
62. Run the staged prompt; approve or correct each contract before the next stage.
63. Implement the schema and seed data, then run constraint tests.
64. Implement ticket create, assign, close, filter and export/import workflows.
65. Run the SLA report and reconcile every dashboard total.
66. Complete ACCEPTANCE.md with test evidence, known limitations and the final prompt history.

Core SQL

```
SELECT status,priority,COUNT(*)
tickets,ROUND(AVG((julianday(COALESCE(closed_at,'now'))-
julianday(created_at))*24),1) avg_hours FROM tickets GROUP BY status,priority
ORDER BY status,priority;
```

Acceptance and troubleshooting

Acceptance: All acceptance tests pass: integrity, parameter binding, CRUD, report reconciliation, export/import and responsive UI.

If it fails: Vibe-coded output drifts across files → freeze interfaces, test after each change and keep a decision/prompt log.

Glossary

Grain — What one row represents.

Bound parameter — A value supplied separately from SQL structure.

Reconciliation — An independent count or amount proving transformation completeness.

Query plan — SQLite's access strategy, inspected with EXPLAIN QUERY PLAN.

Star schema — Facts at a declared grain joined to descriptive dimensions.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg